ТОО ”Колледж Хекслет”

Лабораторная работа №4

Связи между таблицами Internet Shop

Выполнено студентом:

Дубинин Кирилл

Практическое задание

Раздел RO4_3. Связи между таблицами Internet Shop.

Цель: Расширить базу shop_db, создать таблицы пользователей и заказов и практически реализовать три типа связей: • 1:1 • 1:N • M:N

Условие: RO4_3/lesson4/ЛР-4.pdf

Требования задания

• User ↔ User Profile как 1:1;

• User → Orders как 1:N;

• Category → Products как 1:N;

• Orders ↔ Products как M:N;

• внешний ключ user_profiles.user_id;

• внешний ключ orders.user_id;

• внешний ключ products.category_id;

• внешние ключи order_items.order_id и order_items.product_id;

• составной PRIMARY KEY (order_id, product_id);

• CHECK (quantity > 0);

• корректную работу JOIN;

• отклонение несуществующих внешних ключей.

Ход выполнения

User–UserProfile реализует 1:1 через PK/FK user_id; User–Order и Category–Product — 1:N; Order–Product — M:N через order_items. Составной ключ запрещает повтор пары order/product, CHECK запрещает quantity ≤ 0. JOIN соединяет пользователя, заказ и товары. Итог рассчитывается по исторической цене позиции, умноженной на количество.

Проект: RO4_3/lesson4

Запуск: npm install; npm start. Для PostgreSQL параметры подключения задаются в .env по образцу .env.example. Порядок применения SQL и миграций приведён в README.md проекта.

sql/03_relations.sql

```sql
CREATE TABLE users (
    id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE
);
CREATE TABLE user_profiles (
    user_id INTEGER PRIMARY KEY REFERENCES users(id) ON DELETE RESTRICT ON UPDATE CASCADE,
    phone VARCHAR(30),
    address VARCHAR(250)
);
CREATE TABLE orders (
    id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE RESTRICT ON UPDATE CASCADE,
    status VARCHAR(30) NOT NULL DEFAULT 'new',
    created_at TIMESTAMPTZ(6) NOT NULL DEFAULT CURRENT_TIMESTAMP
);
CREATE TABLE order_items (
    order_id INTEGER NOT NULL REFERENCES orders(id) ON DELETE RESTRICT ON UPDATE CASCADE,
    product_id INTEGER NOT NULL REFERENCES products(id) ON DELETE RESTRICT ON UPDATE CASCADE,
    quantity INTEGER NOT NULL CHECK (quantity > 0),
    price INTEGER NOT NULL CHECK (price > 0),
    PRIMARY KEY (order_id, product_id)
);
INSERT INTO users (name, email) VALUES ('Anna', 'anna@example.com'), ('Ivan', 'ivan@example.com'), ('Maria', 'maria@example.com');
INSERT INTO user_profiles (user_id, phone, address) SELECT id, '+7 700 000 00 00', 'Алматы' FROM users;
INSERT INTO orders (user_id, status) SELECT id, 'new' FROM users WHERE email IN ('anna@example.com', 'ivan@example.com');
INSERT INTO order_items (order_id, product_id, quantity, price)
SELECT o.id, p.id, CASE WHEN p.name = 'Mouse' THEN 2 ELSE 1 END, p.price
FROM orders o JOIN products p ON p.name IN ('Keyboard', 'Mouse') WHERE o.user_id =
```

Листинг 1. sql/03_relations.sql (часть 1)

```sql
(SELECT id FROM users WHERE email = 'anna@example.com');
INSERT INTO order_items (order_id, product_id, quantity, price)
SELECT o.id, p.id, 1, p.price FROM orders o JOIN products p ON p.name = 'Keyboard'
WHERE o.user_id = (SELECT id FROM users WHERE email = 'ivan@example.com');
```

Листинг 2. sql/03_relations.sql (часть 2)

sql/relations_queries.sql

```sql
SELECT * FROM users u JOIN user_profiles p ON p.user_id = u.id;
SELECT o.id, u.name, p.name AS product, oi.quantity, oi.price,
       oi.quantity * oi.price AS line_total
FROM orders o JOIN users u ON u.id = o.user_id
JOIN order_items oi ON oi.order_id = o.id JOIN products p ON p.id = oi.product_id
ORDER BY o.id, p.id;
SELECT order_id, SUM(quantity * price) AS total FROM order_items GROUP BY order_id;
SELECT o.* FROM orders o JOIN order_items oi ON oi.order_id = o.id WHERE oi.product_id =
 1;
```

Листинг 3. sql/relations_queries.sql

services/orderService.js

```javascript
import { pool } from '../config/db.js';
export async function getAllOrders() {
    return (await pool.query('SELECT id, user_id AS "userId", status, created_at AS
"createdAt" FROM orders ORDER BY id')).rows;
}
export async function getOrderItems(id) {
    return (await pool.query('SELECT p.id, p.name, oi.quantity, oi.price, oi.quantity *
oi.price AS "lineTotal" FROM order_items oi JOIN products p ON p.id = oi.product_id
WHERE oi.order_id = $1 ORDER BY p.name', [id])).rows;
}
export async function findOrderById(id) {
    return (await pool.query('SELECT id FROM orders WHERE id = $1', [id])).rows[0];
}
```

Листинг 4. services/orderService.js

Проверка результата

Проверено: 05.11.2026 13:38 (Алматы). Node.js v26.12.1.

Приведены фактические результаты проверки. Полный журнал: verification.json.

• Реальный PostgreSQL

• HTTP endpoints

• CRUD сохраняется в PostgreSQL

• UNIQUE и FOREIGN KEY; категории перенесены без потери товаров

• Связи 1:1, 1:N, M:N; CHECK(quantity > 0); составной ключ

HTTP-запросы

GET /api/products → 200

GET /api/products/1 → 200

GET /api/products/999999 → 404

GET /api/products/abc → 400

GET /api/products?category=Accessories&inStock=true&minPrice=10000&maxPrice=50000 → 200

GET /api/products?minPrice=abc → 400

GET /api/products?inStock=maybe → 400

GET /api/products?minPrice=500&maxPrice=100 → 400

POST /api/products → 403

POST /api/products → 400

POST /api/products → 201

PATCH /api/products/6 → 200

DELETE /api/products/6 → 200

GET /api/products/6 → 404

POST /api/products → 409

GET /api/orders/1/items → 200

GET /api/orders/999999/items → 404

GET /api/orders/abc/items → 400

GET /api/unknown → 404

Примеры ответов

POST /api/products → 201

{

```
  "id": 6,
  "name": "Test SSD",
  "price": 65000,
  "categoryId": 1,
  "category": "Accessories",
  "inStock": true
}
```


PATCH /api/products/6 → 200


```
{
  "id": 6,
  "name": "Test SSD",
  "price": 42000,
  "categoryId": 1,
  "category": "Accessories",
  "inStock": false
}
```


DELETE /api/products/6 → 200


```
{
  "id": 6,
  "name": "Test SSD",
  "price": 42000,
  "category_id": 1,
  "inStock": false
}
```


Результат

Проверены обязательные операции и указанные ошибочные сценарии; результаты сохранены вместе с исходным кодом.